



CasMST-KAN: a cascade prediction model with multi-scale temporal convolution and Kolmogorov–Arnold networks

Xia Dan¹ · Zhihao Zhang¹ · Yuan Tian¹ · Di Chen¹

Received: 8 February 2025 / Accepted: 9 February 2026

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2026

Abstract

Predicting information diffusion cascades in social networks is crucial, particularly for forecasting cascade sizes. However, existing methods often struggle to handle the complex dynamics of information spread, especially in terms of multi-scale temporal patterns and intricate node interactions. This paper introduces *CasMST-KAN*, a novel deep learning model designed to address these challenges by integrating advanced techniques for both temporal and spatial modeling. Specifically, we propose a temporal information processing branch that combines *multi-scale temporal convolution (MSTC)* and long short-term memory (LSTM) to effectively capture dynamic patterns across multiple temporal scales. In parallel, Kolmogorov–Arnold networks (KAN) replace traditional MLPs to more flexibly model complex nonlinear node relationships essential to diffusion. Additionally, a ReLU-based multi-scale linear attention mechanism is introduced to enhance representation learning, improving feature extraction across scales. Extensive experiments on multiple real-world datasets demonstrate that CasMST-KAN significantly outperforms state-of-the-art methods in predicting information diffusion cascades, particularly excelling in long-term cascades where capturing temporal dynamics and deep structural dependencies is crucial. Code and datasets are available at <https://github.com/Slip01/CasMST-KAN>.

Keywords Online social networks · Cascade size prediction · Information diffusion · Multi-scale temporal patterns

1 Introduction

With the rapid advancement of social media and communication technologies, the scale and depth of information diffusion have reached unprecedented levels. Platforms like Weibo, Twitter, and YouTube enable users to engage

in cascading forms of information dissemination through likes, comments, and reposts, which facilitate the swift propagation of information across networks [30]. This cascade diffusion plays a pivotal role in applications such as social recommendations [14], rumor blocking [26, 43], and anxiety reduction [32]. However, accurately predicting the scale and trajectory of information diffusion remains highly challenging due to the complex structure of social networks, the dynamic nature of information spread, and the intricate relationships among individual users. Effectively modeling and forecasting these cascading processes offers valuable insights into the mechanisms of information propagation and provides a strong foundation for various social and commercial applications.

The task of predicting information cascades is a prominent topic in social media analysis, which involves predicting the future spread or popularity of a given piece of information based on early cascade data within a specific observation window. Traditional cascade prediction methods generally fall into four categories: diffusion-based models, feature extraction approaches, generative models,

Xia Dan and Yuan Tian have contributed equally to this work.

Xia Dan and Di Chen contributed equally to this work.

✉ Xia Dan
danxia@ccnu.edu.cn

Zhihao Zhang
z2023114419@mails.ccnu.edu.cn

Yuan Tian
tianyuanccnu@mail.ccnu.edu.cn

Di Chen
chendi@mail.ccnu.edu.cn

¹ Faculty of Artificial Intelligence in Education, Central China Normal University, Luoyu Road, Wuhan 430079, Hubei, China

and deep learning-based methods, each addressing unique aspects of cascade dynamics.

(1) *Diffusion model-based methods* [15, 19] simulate information spread using predefined diffusion models, such as the Independent Cascade Model and the Linear Threshold Model. However, these methods struggle to adapt effectively to cascade prediction tasks. (2) *Feature extraction methods* [7, 28] employ feature engineering to identify structural, temporal, and user-specific attributes related to cascades, which are then input into traditional machine learning models for prediction. These methods, however, heavily rely on the quality of handcrafted features. (3) *Generative models* [11, 28, 29, 41] apply statistical theories, like Poisson and Hawkes processes, to model information propagation, but often overlook implicit information within cascade dynamics, leading to subpar predictive performance. (4) *Deep learning-based methods* have shown great promise due to their strong pattern recognition capabilities, which is also the focus of this paper. Most current deep learning approaches adopt variants of RNNs (e.g., GRU [9], LSTM [13]) or graph embedding techniques (e.g., path sampling [3], GNNs [21]) to capture both temporal sequences and graph structures within cascades. To overcome the limitations of traditional RNN- and GNN-based models, several recent efforts have proposed more advanced architectures. Transformer-based models such as CasTformer [31] leverage global spatiotemporal representations and relative positional encoding to improve prediction accuracy. However, the reliance on large Transformer blocks often leads to high computational overhead, which can hinder real-time scalability. Heterogeneous GNNs like HERI-GCN [36] incorporate multiple node types and time features, but their temporal modeling is still based on recurrent units, which limits their ability to capture long-range or multi-scale temporal dependencies. Meanwhile, hierarchical graph methods such as CSN [18] utilize multi-level cascade graphs and second-order neighbor sampling, improving structural modeling at the cost of increased model complexity and less interpretability.

Despite recent advances in information cascade prediction, existing methods still face several limitations:

- *Limited ability to model temporal patterns at multiple scales:* Most approaches rely on standard RNNs (e.g., HERI-GCN) or global self-attention mechanisms (e.g., CasTformer), which may either miss short-term bursts or dilute long-term trends. Capturing both fine-grained and coarse-grained temporal dynamics remains challenging.
- *Inadequate representation of complex, nonlinear relationships:* Existing models often rely on standard MLPs (e.g., in CSN) or shallow attention mechanisms that may fail to capture the intricate, nonlinear dependencies

between diffusion paths and user interactions in cascades.

- *Computational inefficiency in feature extraction across scales:* While powerful, models like CasTformer and CSN are computationally intensive due to large Transformer layers or multi-graph encoding. This limits their practical deployment in large-scale or real-time applications.

To address the key challenges in information cascade prediction—including multi-scale temporal modeling, complex structural dependencies, and efficient feature extraction—we propose a novel deep learning framework, *CasMST-KAN*, which integrates complementary modules in a dual-branch architecture to jointly model structural and temporal dynamics.

At the core of CasMST-KAN is a temporal processing branch that combines multi-scale temporal convolution (MSTC) and long short-term memory (LSTM) networks. MSTC employs parallel dilated convolutions with varying receptive fields to capture temporal dependencies across multiple scales, which is essential for modeling the heterogeneous evolution patterns of cascades, such as early surges and late plateaus. LSTM further complements this by learning long-term sequential dependencies that are difficult to capture via convolution alone. This hybrid design ensures comprehensive temporal modeling, effectively capturing both short-term fluctuations and long-range trends.

In parallel, the structural processing branch begins with graph attention networks (GAT) [34], which model both local and global diffusion patterns based on node interactions in the cascade graph. To further enhance efficiency and contextual focus, we incorporate a lightweight ReLU-based multi-scale linear attention module [2], which offers computationally efficient attention over structural features while maintaining the ability to highlight salient regions.

To unify the outputs from both branches, the extracted representations are fused and passed through a Kolmogorov–Arnold network (KAN) [24], a powerful function approximator with stronger expressiveness than traditional MLPs. KAN enhances the model’s capacity to learn complex, nonlinear interactions involved in cascade propagation.

Compared to previous models that rely on a single type of temporal encoder or shallow structural integration, CasMST-KAN strategically combines multi-scale temporal convolution, sequential modeling, attention-enhanced structural encoding, and expressive nonlinear function approximation. This design exploits the complementary strengths of MSTC, LSTM, and KAN, offering a comprehensive and efficient solution for predicting cascade growth. Extensive experiments on benchmark datasets demonstrate its superior

performance over state-of-the-art methods in both accuracy and scalability.

Our primary contributions are as follows:

- *Temporal information processing branch*: To enhance multi-scale temporal processing, *CasMST-KAN* is designed with a temporal stream processing branch that integrates multi-scale temporal convolution (MSTC) and long short-term memory (LSTM). By leveraging this design, *CasMST-KAN* effectively aggregates information across temporal scales, providing a more comprehensive understanding of cascade dynamics over time.
- *Kolmogorov–Arnold Networks (KAN)*: To better capture nonlinear and high-dimensional relationships among nodes, *CasMST-KAN* leverages Kolmogorov–Arnold Networks as an expressive alternative to standard MLPs. This improves the model’s capacity to represent complex influence patterns in the cascade structure.
- *ReLU-based multi-scale linear attention*: To enhance local information extraction and facilitate multi-scale learning, *CasMST-KAN* employs a lightweight ReLU-based multi-scale linear attention mechanism. This design significantly improving feature extraction and performance across scales.
- *Superior performance on benchmark datasets*: Extensive evaluations on several benchmark datasets demonstrate the effectiveness of *CasMST-KAN*, showing that our model significantly outperforms existing methods in terms of accuracy, underscoring its exceptional capability in information cascade prediction.

The remainder of this paper is organized as follows. Section 2 reviews related work on social media prediction and information cascade prediction. Section 3 introduces the problem formulation and provides formal definitions. Section 4 describes the architecture of the proposed *CasMST-KAN* model in detail. Section 5 presents the experimental setup, datasets, and results. Finally, Sect. 6 concludes the paper and discusses potential directions for future research.

2 Related work

2.1 Cascade prediction

Predicting information cascades in social networks plays a critical role in applications such as event forecasting, public opinion monitoring, and resource planning on social media platforms [1, 10, 40]. We briefly review existing cascade prediction approaches, which can be grouped into four main categories: diffusion model-based methods, feature-based

approaches, generative process approaches, and deep learning-based approaches.

2.1.1 Diffusion model-based methods

Diffusion model-based methods assume that information spreads according to predefined models, such as the independent cascade (IC) [16] and linear threshold (LT) [20] models. These approaches focus on determining node activation states through probabilistic simulations or fixed thresholds. However, the strong assumptions inherent in these models often limit their adaptability to the diverse and dynamic nature of real-world cascades, making them less effective for accurate cascade prediction.

2.1.2 Feature-based approaches

Feature extraction methods aim to identify and leverage high-quality features related to cascades, utilizing traditional machine learning techniques for predictions. Content features [25, 27], such as the number and sentiment of hashtags, URL domains, and subject matter, are often mined from posts and serve as auxiliary inputs. Structural features [7, 12], derived from social networks—such as friendship and reposting dynamics—tend to enhance prediction performance more effectively than content features alone. Temporal features [28, 37], including repost timestamps and time intervals, play a crucial role in capturing the timing aspects of cascades. Additionally, user-related features [7], such as user profiles and behaviors, are combined with these other feature types to improve prediction accuracy. Nonetheless, these methods are limited by the quality of extracted features and the lack of a standardized approach for feature extraction across different domains, which can hinder their generalizability.

2.1.3 Generative process approaches

Generative process methods focus on modeling the temporal dynamics of information diffusion by observing individual events and estimating parameters that maximize the likelihood of these events occurring within a specified time frame. Commonly employed models in this category include the Poisson process [11, 29] and various point processes [28, 41]. These methods are advantageous because they generally require less intensive feature engineering and offer improved interpretability [42]. However, their reliance on time-series data limits their ability to fully leverage the underlying structural information present in cascade networks. Consequently, while generative methods can effectively capture temporal patterns, they often fall short in accounting for the complex interactions and relationships

between nodes. This limitation raises concerns about their predictive accuracy, especially in intricate diffusion scenarios where the nuances of network structure play a critical role.

2.1.4 Deep learning-based approaches

In recent years, deep learning methods for popularity prediction have increasingly focused on integrating diverse types of information, including temporal dynamics, node and structural representations, and content features. For instance, models like DeepHawkes [3] employ a Hawkes process to offer interpretable insights into information cascades. Meanwhile, the emergence of graph neural networks (GNNs) has led to innovations such as CasCN [5], which utilizes cascade subgraphs to model structural relationships. CasFlow [38] introduces a variational autoencoder to assess the uncertainty of cascades, while TempCas [33] relies on heuristic sampling of critical paths, supplementing this with handcrafted features to mitigate the shortcomings of sampling.

Despite their strengths in illustrating subgraph dynamics, these models often fall short in capturing the comprehensive cascade effects of information propagation. For instance, CouledGNN [4] highlights interactions between node activation states through global propagation graphs, and CasGCN [39] investigates the bidirectional nature of cascades by employing both incoming and outgoing adjacency matrices. However, the tendency to model each node individually can compromise efficiency and overall predictive performance.

To better model uncertainty and irregular temporal patterns, CasDO [8] introduces probabilistic diffusion models and neural ODEs, enabling continuous-time modeling. Nonetheless, the use of backward-forward diffusion processes significantly increases model complexity and training cost. Similarly, CasTformer [31] leverages a Transformer backbone with spatiotemporal positional encodings to model interpersonal influence. While effective, its large Transformer blocks introduce high computational overhead, making it less suitable for real-time or resource-constrained applications.

Multi-source and heterogeneous relationships are further emphasized in HERI-GCN [36], which builds a heterogeneous cascade graph across time intervals and integrates RNNs into GCNs for spatiotemporal learning. HeDAN [17] constructs a heterogeneous diffusion graph incorporating user-message relations (friendship, interaction, interest), and utilizes hierarchical attention—node-level GATs and semantic-level multi-head attention—to learn refined representations of user preferences and behavior.

Incorporating social structures beyond individual cascades, Cascade Social Net [18] combines cascade graphs, inter-cascade links, and user social networks into a unified multi-view learning framework, demonstrating the importance of broader social context in cascade prediction.

In terms of temporal modeling, DeepHawkes [3] integrates a nonparametric time decay function within recurrent frameworks, while DFTC [22] utilizes time-series data to capture both short-term outbreaks and long-term trends using Convolution-1D and LSTM networks. TempCas [33] enhances this approach by blending short- and long-term dynamics but frequently overlooks the potential of global and local temporal insights.

Despite these advances, most existing models either suffer from limited temporal granularity or insufficient non-linear representation capacity. To address these limitations, we introduce our model, *CasMST-KAN*, which effectively integrates a specially designed temporal information processing branch, Kolmogorov–Arnold Networks (KAN), and a ReLU-based multi-scale linear attention mechanism. The temporal information processing branch, incorporating multi-scale temporal convolution (MSTC) and long short-term memory (LSTM), captures multi-scale temporal patterns, providing a nuanced understanding of temporal dynamics in cascades. Simultaneously, KAN models intricate interactions between nodes, enhancing *CasMST-KAN*'s ability to represent complex structural relationships. Our approach leverages both temporal and structural information more comprehensively, leading to improved prediction accuracy and efficiency in cascade size forecasting.

2.2 Model component motivation and related work

To enhance the interpretability and contextual grounding of our proposed *CasMST-KAN* framework, we provide a detailed overview of its core components—multi-scale temporal convolution (MSTC), ReLU-based multi-scale linear attention, and the Kolmogorov–Arnold networks (KAN). This section positions each component within the broader literature, clarifying its motivation and relation to prior work.

2.2.1 Multi-scale temporal convolution (MSTC)

Temporal modeling is essential for capturing the evolution of information cascades over time. Multi-scale temporal convolution (MSTC) was initially introduced in TopoGDN [23] for multivariate time-series anomaly detection. MSTC applies multiple 1D convolutions with varying dilation rates to extract both local and global temporal features from time-series inputs. This design enables the model to capture fine-grained temporal dependencies over extended windows,

addressing the conflict between short-term patterns and long-range trends. This design enables our model to better understand diverse cascade dynamics across varying time scales.

2.2.2 ReLU-based multi-scale linear attention

Traditional attention mechanisms, such as softmax-based self-attention, offer high expressive power but suffer from quadratic complexity, limiting scalability for long sequences and large-scale data. To address this, we draw inspiration from EfficientViT [2], which introduces a hardware-friendly multi-scale linear attention mechanism. Instead of relying on softmax normalization, this design adopts linear operations combined with ReLU activations to approximate global context aggregation in a more efficient way. The ReLU-based multi-scale linear attention not only reduces computational overhead but also supports multi-scale learning, making it well-suited for large cascade modeling.

2.2.3 Kolmogorov–Arnold networks (KAN)

Kolmogorov–Arnold networks (KAN) [24] are a novel neural network architecture based on the Kolmogorov–Arnold representation theorem, which asserts that any multivariate continuous function can be decomposed into a composition of univariate functions and summations. Unlike traditional MLPs that apply fixed nonlinearities to each node, KANs replace weights with learnable univariate functions (e.g., splines), while keeping nodes purely additive. This structure allows KANs to serve as more flexible and interpretable function approximators. In our work, we adopt KANs as a replacement for MLPs to improve the model's capacity in capturing complex nonlinear relationships within cascade dynamics, leading to better predictive performance without increasing architectural depth.

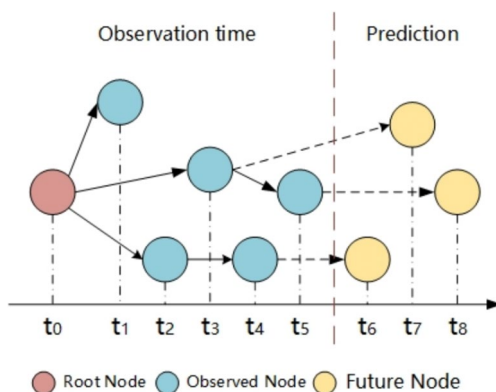


Fig. 1 An illustration of the information cascade

3 Preliminaries

Information cascade prediction aims to forecast how widely a piece of content (e.g., a tweet, post, or video) will eventually spread on a social network, based on its early propagation patterns. This is a crucial task in applications such as misinformation control, viral marketing, and trend forecasting.

An *information cascade* describes a process where users consecutively share content, forming a directed graph that evolves over time. In this work, we model each cascade as a graph with both structural and temporal attributes. The prediction task focuses on estimating the future growth of a partially observed cascade.

Figure 1 illustrates a typical cascade graph, where nodes represent users and edges represent retweet or repost relationships over time.

Definition 1 (Cascade graph) Let there be n posts, $P = \{p_c \mid c \in [1, n]\}$. For each post p_c , the corresponding cascade graph is denoted by $G_c = (V_c, E_c, T_c)$.

- V_c is the set of nodes involved in the cascade during the observation time T ,
- A directed edge $(v_i, v_j) \in E_c$ indicates that node v_j retweets from node v_i ,
- A tuple $(v_i : t_i) \in T_c$ denotes the time t_i elapsed between the original post and the retweet by node v_i .

Definition 2 (Growth size) The growth size of a cascade is defined as the increase in the number of nodes in the cascade over the observation time window T , after an additional time interval Δt has passed. Based on Definition 1, we obtain the cascade graph $G_c = (V_c, E_c, T_c)$, where $T_c < T$. The task is to predict the growth size ΔS_i of a cascade after a given time interval Δt . The growth size is defined as:

$$\Delta S_i = |V_{T+\Delta t}^i| - |V_T^i|$$

$|V_{T+\Delta t}^i|$ and $|V_T^i|$ represent the number of nodes in the cascade at times $T + \Delta t$ and T , respectively.

4 Model

In this section, we provide a detailed description of the CasMST-KAN model proposed in this paper. As illustrated in Fig. 2, CasMST-KAN consists of the following major components: (1) Cascade sample input: cascade samples are processed through an algorithmic sampling method to extract key components. (2) Structural information

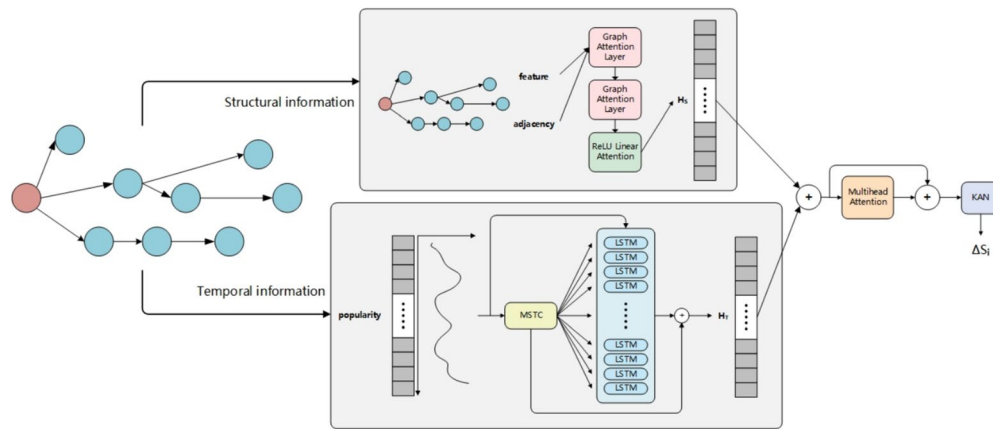


Fig. 2 The framework of CasMST-KAN

processing branch: this branch utilizes two layers of graph attention networks (GAT) to extract structural information from the graph. A ReLU-based multi-scale linear attention mechanism is further employed to enhance the extraction of local structural features. (3) Temporal information processing branch: multi-scale temporal convolution (MSTC) is used initially to capture multi-scale temporal information, which is then passed through an LSTM for further processing. (4) Feature fusion: after processing the structural and temporal information, a multi-head attention mechanism is employed to effectively fuse these features. (5) Prediction: the fused features are then passed to the Kolmogorov–Arnold networks (KAN) for the final prediction. In the following Algorithm 1, we present a simplified pseudocode outlining the data processing flow of CasMST-KAN.

```

1: Input: adjacency, features, temporal data
2: Output: predicted cascade size
3: 1. Structural information processing
4:  $h \leftarrow \text{GAT1}(\text{features}, \text{adjacency})$ 
5:  $h \leftarrow \text{ReLU}(h)$ 
6:  $h \leftarrow \text{GAT2}(h, \text{adjacency})$ 
7:  $h \leftarrow \text{ReLULinearAttention}(\text{reshape}(h))$ 
8: 2. Temporal information processing
9:  $\text{interval} \leftarrow \text{MSTC}(\text{temporal data}) + \text{temporal data}$ 
10:  $\text{interval} \leftarrow \text{LSTM}(\text{interval}) + \text{interval}$ 
11:  $\text{interval} \leftarrow \text{ReLULinearAttention}(\text{interval}) + \text{interval}$ 
12: 3. Feature fusion with multi-head attention
13:  $\text{fused} \leftarrow \text{MultiHeadAttention}(\text{concat}(h, \text{interval}))$ 
14: 4. Final fusion and prediction
15:  $\text{output} \leftarrow \text{MLP}(\text{fused})$ 
16: return output

```

Algorithm 1 Simplified data flow of CasMST-KAN

4.1 Input of cascade samples

The first step in CasMST-KAN is the input of cascade samples. Since the subsequent stages rely on graph neural networks (GNNs) to extract structural information from the

cascade, the large scale of the cascade can result in significant computational costs for the GNN. To mitigate this, it is essential to efficiently sample key nodes within the cascade, thereby reducing the number of nodes while preserving the critical information of the graph. This approach enables more efficient computation and training.

The graph sampling process in this section is adapted from the algorithm proposed by Cheng et al. [6]. Given a cascade graph G_c and its adjacency matrix A_c , we first compute the outdegree $D = \{d_i, i \in [1, n]\}$, where the outdegree d_i is defined as:

$$d_i = \log_2 \left(\sum_{j=1}^N a_{ij} \right)$$

Here, N denotes the total number of nodes, a_{ij} represents the elements of the adjacency matrix, and d_i is the logarithmically scaled outdegree of node i . Since nodes with larger outdegrees typically have a greater influence on the cascade propagation, we sort and filter the nodes according to the following rules:

- **Rule 1:** Sort the nodes in descending order by outdegree D .
- **Rule 2:** If multiple nodes have the same outdegree, sort them by their occurrence time, from latest to earliest.

To focus CasMST-KAN on critical nodes while minimizing computational overhead, we sample the top K nodes based on the sorted outdegree vector. However, this sampling process may result in the exclusion of some parent nodes. To address this issue, we identify the nearest ancestor node that has not been filtered out and use it as the parent node.

The complete graph sampling process is shown in Algorithm 2.

```

1: Input: A cascade network  $C_c = (V_i, E_i, T_i)$ , degree vector  $D$  of  $G_c$ ; the maximum
   number of nodes  $K$ 
2: Output: Sampled cascade network  $C_{sampled}$  and out-degree vector  $D_{sampled}$ 
3:  $V_c, E_c, T_c \leftarrow C_c$ 
4: if  $|V_c| < K$  then return  $C_c, D$ 
5: end if
6:  $D_{sorted}, V_{sorted} \leftarrow \text{sort}(D, V_c, T_c)$  ▷ Sort by Rule 1 and Rule 2
7:  $D_{sampled}, V_{sampled} \leftarrow \text{selectTop}(D_{sorted}, V_{sorted}, K)$ 
8:  $E_{sampled} \leftarrow \emptyset, T_{sampled} \leftarrow \emptyset$ 
9: for each  $\{v_i, v_j\} \in E_c$  do
10:   if  $v_j \notin V_{sampled}$  then
11:     continue
12:   else if  $v_i \in V_{sampled}$  then
13:      $E_{sampled} \leftarrow E_{sampled} \cup \{v_i, v_j\}$ 
14:   else
15:      $v_k \leftarrow \text{findAncestor}(v_i, E_c, V_{sampled})$ 
16:      $E_{sampled} \leftarrow E_{sampled} \cup \{v_k, v_j\}$ 
17:   end if
18: end for
19: for each  $v \in V_{sampled}$  do
20:    $T_{sampled} \leftarrow T_{sampled} \cup (v : T(v))$ 
21: end for
22:  $C_{sampled} = (V_{sampled}, E_{sampled}, T_{sampled})$ 
23: return  $C_{sampled}, D_{sampled}$ 

```

Algorithm 2 Graph sampling

4.2 Structural information processing branch

In the structural information processing branch, the input node features and adjacency matrix are processed using the graph attention layer (GAT), which enhances the propagation of structural information and captures context-sensitive relationships, crucial for accurate cascade prediction. To further improve the representation of cascade dynamics, a multi-scale ReLU-based multi-scale linear attention mechanism is applied, enabling CasMST-KAN to capture diverse structural patterns and better model the spread and growth of the cascade.

4.2.1 Graph attention layer (GAT)

Node feature transformation: The input node feature matrix h undergoes a linear transformation via matrix multiplication with the weight matrix W , yielding the transformed node feature matrix Wh . This transformation projects node features into a new space, facilitating the extraction of latent structural relationships:

$$Wh = h \cdot W. \quad (1)$$

Attention coefficient calculation: To compute the unnormalized attention scores e_{ij} between each node pair i and j , we

employ the function `_prepare_attentional_mechanism_input`, which constructs the pairwise concatenated features $[h_i || h_j]$ as input to the attention mechanism. Here, h_i and h_j represent the feature vectors of nodes i and j , respectively, and $||$ denotes the concatenation operation. Specifically, the score e_{ij} is obtained by applying a shared learnable weight vector a through a dot product operation, followed by a LeakyReLU activation function:

$$e_{ij} = \exp(\text{LeakyReLU}(a^T [h_i || h_j])). \quad (2)$$

Masking and normalization: The adjacency matrix adj is applied as a mask on the attention coefficients, retaining only those corresponding to actual edges in the graph. Non-edge positions are assigned a small value ($-9e15$) to make their softmax values negligible. Here, $N(i)$ denotes the set of neighbors of node i . The attention coefficients are then normalized using the softmax function, ensuring that the coefficients sum to one:

$$\alpha_{ij} = \frac{e_{ij}}{\sum_{k \in N(i)} e_{ik}}. \quad (3)$$

Weighted aggregation: The normalized attention coefficients are then used to perform a weighted sum of the node features Wh , resulting in the updated node feature matrix

h' . This weighted aggregation enables each node's feature representation to incorporate relevant information from its neighbors according to the learned attention distribution:

$$h'_i = \sum_{j \in N(i)} \alpha_{ij} \cdot Wh_j. \quad (4)$$

Activation and output: Finally, based on the `concat` parameter setting, the output of multi-head attention is either concatenated or summed across heads. The resulting output is passed through a ReLU activation function to introduce non-linearity, enhancing CasMST-KAN's capacity to capture complex relationships within the graph:

$$h'_{\text{GAT, ReLU}} = \text{ReLU}(h'_i). \quad (5)$$

4.2.2 ReLU-based multi-scale linear attention mechanism

To further improve the expressiveness of graph embedding features, we apply a multi-scale linear attention mechanism with ReLU activation.

Multi-scale Q , K , V generation: The graph embedding features h' are first passed through a 2D convolution layer to generate multi-scale query, key, and value vectors (q , k , and v):

$$qkv = \text{Conv2d}(h'). \quad (6)$$

We apply convolutions at different scales using an aggregation module to generate additional contextual representations. The multi-scale outputs are concatenated along the channel dimension, forming a comprehensive representation `multi_scale_qkv`.

Lightweight linear attention computation: The concatenated features are decomposed into q , k , and v matrices, followed by ReLU activation for both q and k :

$$q = \text{ReLU}(q), \quad k = \text{ReLU}(k). \quad (7)$$

The attention output H_S is computed using lightweight linear attention:

$$H_S = \frac{q \cdot (k^\top \cdot v)}{\text{out}_{\text{norm}}} \quad (8)$$

out_{norm} is a normalization factor introduced to ensure numerical stability during the attention computation.

Advantages over traditional attention: Unlike conventional softmax-based attention—which requires computing full pairwise similarities and incurs $O(n^2)$ time complexity—this ReLU-based linear formulation only performs

matrix multiplications with non-negative activations, reducing the cost to $O(n)$. This makes it more scalable for processing large graph structures.

Moreover, ReLU-based attention promotes sparse activations, enabling sharper focus on structurally relevant nodes and suppressing noisy interactions—particularly beneficial in real-world cascade graphs with sparse, localized diffusion patterns.

Practical scenario example: In a typical retweet cascade, attention should emphasize nodes in immediate subtrees (e.g., friends-of-friends) rather than global nodes. Our ReLU-based linear attention naturally emphasizes such local context, while standard attention may dilute focus across the entire graph due to softmax normalization.

By integrating GAT with the ReLU-based multi-scale linear attention mechanism, the structural information processing branch effectively captures both local and global structural patterns within the graph.

4.3 Temporal information processing branch

The temporal information processing branch utilizes the multi-scale temporal convolution (MSTC) module and long short-term memory (LSTM) layers to capture temporal dependencies and learn multi-scale features from time series data. This is essential for modeling dynamic temporal trends and understanding periodic patterns and short-term fluctuations in the cascade dynamics.

Multi-scale temporal convolution (MSTC) In this branch, the input `interval_popularity` is first processed by the multi-scale temporal convolution (MSTC) module. The purpose of the MSTC module is to extract features at different temporal scales, enabling CasMST-KAN to capture both local and global patterns in the time series. The MSTC module consists of the following submodules:

1. **Multi-dilation convolution branches:** The MSTC module performs multi-scale analysis by employing several convolutional branches, each with a different dilation rate, thereby increasing the receptive field. This allows CasMST-KAN to capture both short- and long-term dependencies effectively. Specifically, for each dilation rate d in `dilations` = [1, 2, 3, 4], an independent convolutional operation is applied.

These dilation rates were empirically chosen to provide a balanced range of temporal receptive fields. A smaller rate (e.g., $d = 1$) captures local, fine-grained temporal patterns, while larger rates (e.g., $d = 4$) allow the model to perceive more global, long-range dependencies. This choice enables the MSTC module to extract complementary features at multiple scales without excessively increasing the computational burden or losing resolution.

To ensure alignment of the convolutions across branches, the kernel size k and padding size p are adjusted as follows:

$$p = \frac{(k-1) \times d}{2}. \quad (9)$$

The convolution operation for each branch is computed as:

$$y^{(d)} = \text{ReLU}(\text{BatchNorm}(\text{Conv2d}(x, k, d, p))). \quad (10)$$

Here, $y^{(d)}$ represents the output of the branch with dilation rate d , x denotes the input feature map, and p is the padding size. The combination of different dilation rates enhances CasMST-KAN's ability to model temporal heterogeneity in cascade evolution.

2. Max-pooling and 1×1 convolution branches: In addition to the dilation convolution branches, the MSTC module includes a max-pooling branch and a 1×1 convolution branch. The max-pooling branch extracts significant features within local time windows, helping to reduce noise and highlight important information. The 1×1 convolution performs feature refinement without altering the temporal context. The outputs of these branches are computed as follows:

$$y_{\max} = \text{ReLU}(\text{BatchNorm}(\text{MaxPool2d}(\text{Conv2d}(x)))), \quad (11)$$

$$y_{1 \times 1} = \text{Conv2d}(x, 1 \times 1). \quad (12)$$

These two branches focus on different feature extraction strategies, effectively extending the MSTC module's capacity to capture both fine-grained and high-level temporal features.

3. Output aggregation and residual connection: The outputs from all branches are aggregated to form a unified multi-scale feature representation. The final feature representation y_{out} is obtained by averaging the outputs from all branches:

$$y_{\text{out}} = \frac{\sum_{i=1}^N y^{(i)} + y_{\max} + y_{1 \times 1}}{N + 2}. \quad (13)$$

If `residual` is set to `True`, the original input x is added back to the output y_{out} to retain the initial temporal information. This residual connection helps mitigate the vanishing gradient problem in deep networks and strengthens the original features. The final output of the MSTC module is:

$$y_{\text{MSTC}} = y_{\text{out}} + x. \quad (14)$$

The MSTC output contains multi-scale temporal features, which provide a rich feature representation for the subsequent LSTM layer.

Long short-term memory (LSTM) While MSTC excels at capturing multi-scale local temporal patterns via dilated convolutions, it has limited capacity to model long-range temporal dependencies due to its finite receptive field. To address this limitation, we integrate a two-layer long short-term memory (LSTM) module after MSTC to enhance the modeling of global temporal dynamics. The LSTM module effectively maintains memory over extended sequences, enabling the model to capture dependencies that span across distant time steps.

Formally, the output from the MSTC module is reshaped and fed into the LSTM:

$$h_t, c_t = \text{LSTM}(y_{\text{MSTC}}). \quad (15)$$

The output of the LSTM, denoted as y_{LSTM} , provides enriched long-term temporal representations. To preserve the localized features learned by MSTC while incorporating the global context from LSTM, we apply a residual connection:

$$H_T = y_{\text{MSTC}} + y_{\text{LSTM}}. \quad (16)$$

This sequential combination allows CasMST-KAN to benefit from both localized short-term features and global long-term dependencies, enhancing its ability to model complex temporal evolution in information cascades.

4.4 Feature fusion

The feature fusion module integrates graph embedding features and temporal features to generate a unified representation that captures both spatial and temporal dependencies.

First, the graph embedding features h are concatenated with the temporal features (e.g., time vectors) along the feature dimension. This step enriches the representation by incorporating temporal information:

$$H_{\text{fused}} = \text{Concat}(H_s, H_t, \text{dim} = 2). \quad (17)$$

Next, the combined features are passed through a multi-head attention mechanism to refine the representation. The attention mechanism enables CasMST-KAN to focus on the most relevant parts of the combined features, effectively enhancing the overall feature representation:

$$H_{\text{att}, _} = \text{MHA}(H_{\text{fused}}, H_{\text{fused}}, H_{\text{fused}}). \quad (18)$$

Finally, the output from the attention mechanism is added back to the original features in a residual fashion. This residual connection helps retain the original feature information while allowing the attention mechanism to enhance the feature representation:

$$H_{\text{refined}} = H_{\text{att}} + H_{\text{fused}}. \quad (19)$$

This process yields a comprehensive representation that captures both spatial and temporal dynamics, serving as the input for the final prediction module.

4.5 Prediction

The prediction module utilizes the refined features from the fusion module as input to the Kolmogorov–Arnold network (KAN), which is designed to model complex nonlinear relationships and high-order feature interactions, thereby improving the accuracy of cascade size prediction.

Specifically, the refined feature representation H_{refined} is passed through a series of KAN layers, where each layer performs a nonlinear transformation based on learned functional bases:

$$h_{\text{KAN}} = \text{KANLayer}(H_{\text{refined}}). \quad (20)$$

These layers progressively enhance the feature representation by capturing intricate dependencies within the data. Finally, the transformed representation is mapped to a scalar output indicating the predicted cascade size.

By integrating multi-head attention for feature fusion and KAN for nonlinear prediction, CasMST-KAN effectively captures both relational structure and complex temporal dynamics, leading to more accurate forecasting of cascade growth.

5 Experiments

In this section, we evaluate the performance of the proposed CasMST-KAN model using two real-world datasets, comparing it against several state-of-the-art benchmark models. Additionally, we conduct ablation studies and provide a detailed description of the experimental setup and parameters.

5.1 Datasets

Sina Weibo. The Sina Weibo dataset, sourced from Cao et al. [3], is derived from a prominent Chinese microblogging platform comparable to Twitter. This dataset contains posts published over a 24-h period on June 1, 2016. Each post is

associated with metadata, such as timestamps, user information, and repost relationships, offering rich contextual information for cascade modeling. The dataset is particularly valuable for studying short-term information propagation due to its dense activity and rapid interaction patterns.

Twitter. The Twitter dataset, collected by Weng et al. [35], consists of public English tweets published between March 24 and April 25, 2012. It includes tweet texts, timestamps, and retweet information, providing a basis for analyzing long-term adoption dynamics on a global scale.

For the Weibo dataset, observation times T are set to 0.5 h, 1 h, 2 h, and 1 day, reflecting the platform's high temporal granularity. In contrast, the Twitter dataset uses observation times of 1 day, 2 days, and 3 days, aligning with its slower propagation dynamics. The prediction time for the Weibo dataset is fixed at 24 h, while for the Twitter dataset, it is set to 32 days, allowing the capture of both short- and long-term cascade growth. Consistent with prior studies, we exclude cascades where $|V_c| < 10$, ensuring the focus remains on significant propagation events.

To account for diurnal rhythms and ensure data quality, we impose additional constraints. For the Weibo dataset, only posts published between 8:00 a.m. and 6:00 p.m. are included, guaranteeing at least 6 h for retweets to accumulate. For the Twitter dataset, we include tweets published before April 10, ensuring a minimum of 15 days for adopter growth. These preprocessing steps help mitigate temporal biases and ensure consistency in evaluation.

Both datasets are randomly split into training (70%), validation (15%), and test (15%) sets to ensure fair model evaluation. The splitting process maintains the distribution of cascade sizes across subsets to prevent sampling biases. Additionally, Table 1 provides detailed statistics, including the total number of cascades, average cascade sizes, and other relevant metrics, offering a comprehensive overview of the datasets.

5.2 Baselines and settings

We evaluate our proposed model against seven state-of-the-art baseline methods commonly used for cascade size prediction. Some of these methods have been adapted to better align with the task requirements. Below is a brief description of each baseline method:

Feature-linear and feature-deep [7]. These methods extract predictable features from prior studies and use them as inputs for a linear regression model (Feature-Linear) and a fully connected deep neural network (Feature-Deep) to predict cascade size growth.

DeepHawkes [3]. This model bridges the gap between feature-based and generative approaches by integrating the

Table 1 Descriptive statistics of two datasets

Dataset	Ori. cascade	Avg. path length	Avg. popularity	0.5 h/1 day			1 h/2 days			2 h/3 days		
				Train	Val	Test	Train	Val	Test	Train	Val	Test
Weibo	119,313	1.217	171.098	21,461	4598	4598	27,353	5860	5860	32,943	7059	7059
Twitter	88,440	1.201	142.672	9640	2065	2065	12,740	2729	2729	15,777	3380	3380

interpretability of the Hawkes process with the predictive power of deep learning.

CasCN [5]. As the first graph neural network (GNN)-based cascade prediction framework, CasCN leverages both structural information and sub-cascade graphs, incorporating cascade Laplacian matrices. It achieves improved prediction accuracy and computational efficiency without relying on hand-crafted features.

CasFlow [38]. This model introduces a probabilistic framework based on hierarchical cascade normalizing flows. It effectively captures both global and local structures of cascades while modeling uncertainties in the diffusion process.

TempCas [33]. TempCas samples critical paths within cascades and employs handcrafted features to mitigate structural information loss. It incorporates LSTM and attention-based CNNs to model long- and short-term temporal dynamics.

CasSampling [6]. CasSampling is a deep learning framework designed for popularity prediction by sampling critical parts of cascade graphs, thereby improving computational efficiency. It incorporates node outdegree and global propagation time flow to address the loss of structural and temporal information caused by sampling.

These baselines offer a diverse set of methodologies, ranging from feature-based linear models to advanced neural architectures, providing a comprehensive evaluation of the proposed model.

5.3 Experimental settings

In our experiments, we adopt three commonly used regression metrics to evaluate model performance: mean squared logarithmic error (MSLE) and mean absolute percentage error (MAPE). Their definitions are as follows:

$$\text{MSLE} = \frac{1}{N_t} \sum_{p=1}^{N_t} (\log_2 \hat{y}_p - \log_2 y_p)^2, \quad (21)$$

$$\text{MAPE} = \frac{1}{N_t} \sum_{p=1}^{N_t} \left| \frac{\log_2 \hat{y}_p - \log_2 y_p}{\log_2 y_p} \right|. \quad (22)$$

Here, y_p represents the ground truth value, $\hat{y}_p$ denotes the predicted value, and N_t is the size of the test samples.

We choose MSLE as our primary evaluation metric because it emphasizes relative errors in logarithmic space, which is well-aligned with the power-law or long-tail distribution often observed in cascade sizes. MSLE penalizes underestimation and overestimation more fairly when the

true values span multiple orders of magnitude, which is typical in cascade prediction tasks.

We additionally report MAPE, as it provides an interpretable measure of the average percentage error. Compared with metrics like RMSE or MAE, which focus on absolute scale errors, MAPE and MSLE are more appropriate when both small and large cascades must be evaluated proportionally, rather than dominated by large-scale cascades.

Therefore, the combination of MSLE and MAPE offers a balanced and informative view of model performance on heterogeneous cascade size distributions, which better reflects real-world application needs. In Tables 2–5, bold values indicate the best performance for each metric among all compared methods.

The experiments were conducted on a PC equipped with an Intel i5-12400F processor, an NVIDIA GTX 3060Ti GPU, and 16 GB of memory. The implementation of CasMST-KAN is based on PyTorch 2.3.1.

Baseline methods are implemented as described in their respective papers. CasMST-KAN contains 2 layers of graph attention networks (GAT), with hidden dimensions set to 512 and an output dimension of 8 for the node features. For the MSTC module, the kernel size is set to 5. The ReLU-based multi-scale linear attention module uses scales {3, 5}. For the prediction layer, the KAN module utilizes hidden dimensions of {192, 64, 32}, with a grid size of 4 and a spline order of 3. The optimizer used is Adam. We employed Optuna for hyperparameter tuning, optimizing the learning rate and weight decay. The batch size is set to 64, and training is conducted for 150 epochs. During training, we applied early stopping with a patience of 15 epochs, meaning training halts if no improvement in validation performance is observed over 15 consecutive epochs.

5.4 Comparison with baselines

We conducted each experiment 20 times and reported the average results, as shown in Table 2. All experiments were conducted with fixed random seeds and deterministic settings. Therefore, the results are identical across runs, and standard deviations are not applicable. The results reveal several key observations about the performance of CasMST-KAN and other baseline models:

Superior performance of CasMST-KAN: CasMST-KAN consistently outperforms all baseline models across the Weibo and Twitter datasets. It achieves an average improvement of 6.5–9.8% in MSLE and SMAPE compared to the best baseline (CasSampling). For example, on the Weibo dataset, CasMST-KAN achieves an MSLE of 3.169 at the 3-day interval, outperforming CasSampling's MSLE of 3.433. These results demonstrate CasMST-KAN's ability to effectively integrate multi-scale temporal convolution,

Table 2 Performance comparison of different models on Weibo and Twitter datasets

Model	Weibo						Twitter					
	0.5 h			1 h			2 h			1 Day		
	MSLE	SMAPE	MSLE	MSLE	SMAPE	MSLE	MSLE	SMAPE	MSLE	MSLE	SMAPE	MSLE
Feature-Linear	3.025	0.305	2.653	2.653	0.323	2.451	0.332	0.698	6.729	9.123	0.632	5.833
Feature-Deep	2.891	0.281	2.612	2.612	0.319	2.332	0.311	0.669	6.330	7.801	0.599	5.439
DeepHawkes	2.674	0.277	2.538	2.538	0.303	2.312	0.302	0.635	5.085	6.874	0.545	4.281
CasCN	2.660	0.275	2.613	2.613	0.323	2.452	0.310	0.638	5.438	7.121	0.560	4.482
CasFlow	2.418	0.247	2.298	2.298	0.257	2.003	0.281	0.625	5.143	6.989	0.552	4.102
TempCas	2.332	0.243	2.219	2.219	0.248	2.001	0.278	0.611	4.332	6.232	0.525	3.680
CasSampling	2.194	0.227	2.113	2.113	0.243	1.883	0.276	0.594	4.125	5.908	0.512	3.433
CasMST-KAN	2.080	0.219	2.061	2.061	0.243	1.839	0.273	0.560	4.012	5.585	0.477	3.169

Bold values indicate the best results among all compared methods

Kolmogorov–Arnold networks, and ReLU-based multi-scale linear attention to comprehensively capture both temporal dynamics and structural diversity.

Computational efficiency: Due to hardware differences, we did not reimplement the training times or parameter counts of baseline models. The parameter sizes of CasCN (210 M), CasFlow (11 M), and TempCas (12 M) are cited from [6]. Our CasMST-KAN model, with only 2.16M parameters, was trained on the Weibo dataset (1-h prediction task) using an Intel i5-12400F CPU and an NVIDIA GTX 3060Ti GPU, requiring approximately 14 min. These results highlight the lightweight design and efficiency of our model.

Scalability discussion: Although we did not conduct experiments on larger-scale datasets due to hardware limitations, CasMST-KAN was designed with scalability in mind. Its modular structure—featuring multi-scale temporal convolution and linear attention—allows for efficient computation with relatively low memory usage. These components scale well with input sequence length and node count, making the model suitable for large-scale cascade prediction tasks. Further scalability evaluation on larger datasets remains a promising direction for future work.

Challenges in social media cascade prediction: Predicting cascade sizes on social media datasets, such as Weibo and Twitter, is inherently challenging due to shorter repost intervals, larger cascade sizes, and diverse user behaviors. As a result, MSLE values on these datasets are generally higher compared to more structured scientific datasets, reflecting the complexity of modeling real-world cascade behaviors.

Strengths and limitations of baselines:

- **Feature-linear and feature-deep:** These models perform reasonably well by incorporating temporal and structural features, with feature-linear achieving an MSLE of 5.833 on Twitter at the 3-day interval. However, their reliance on handcrafted features and domain knowledge limits scalability and adaptability to complex cascade dynamics.
- **DeepHawkes:** By independently modeling diffusion paths, DeepHawkes effectively captures temporal and influence relationships, achieving an MSLE of 4.281 on Twitter at the 3-day interval. However, it lacks the capacity to capture holistic cascade dynamics, which limits its performance relative to CasMST-KAN.
- **TempCas and CasSampling:** These models focus on temporal patterns, with TempCas leveraging time-series features and CasSampling employing sampling techniques. While competitive, they fall short of CasMST-KAN in modeling both structural and temporal interactions comprehensively.

- **CasCN:** Despite employing GCN-RNN structures to model cascade dynamics, CasCN struggles to fully explore the diverse relationships within cascades, resulting in moderate performance.

In summary, CasMST-KAN demonstrates its superiority by addressing the limitations of existing baselines through the innovative integration of multi-scale temporal convolution and attention mechanisms. It achieves robust and effective cascade size prediction across diverse social media scenarios.

5.5 Ablation study

To thoroughly evaluate the contribution of each component in CasMST-KAN, we conducted an ablation study by systematically removing or replacing key parts of CasMST-KAN, as shown in Table 3. Specifically, we considered the following variants of our model:

CasMST-KAN w/o time flow: In this variant, we removed the multi-scale temporal convolution (MSTC) and the LSTM layers responsible for capturing the temporal dynamics of the information cascade. Without these components, CasMST-KAN loses the ability to capture temporal propagation trends and changes in cascade growth over time. The remaining part of CasMST-KAN relies solely on the graph structure and node embeddings for prediction. We anticipated that this would lead to suboptimal performance, particularly in scenarios where time plays a critical role in the diffusion process.

CasMST-KAN w/o ReLU-based multi-scale linear attention: In this experiment, we completely removed the ReLU-based multi-scale linear attention module, which is crucial for capturing cross-dimensional dependencies between node features and temporal information. By excluding this module, we aimed to evaluate the significance of ReLU-based multi-scale linear attention in enhancing CasMST-KAN's ability to capture intricate relationships between temporal and structural features within the cascade graph. This setup tests whether the inclusion of a non-linear attention mechanism contributes to better feature integration and overall prediction performance.

CasMST-KAN w/o KAN: In this variant, we replaced the Kolmogorov–Arnold networks (KAN) with a standard multi-layer perceptron (MLP) for learning nonlinear mappings. The KAN is designed to approximate complex nonlinear functions by leveraging adaptive, learnable univariate functional transformations on edges, enabling it to capture high-order dependencies in the cascade data. By substituting KAN with a basic MLP, this experiment aims to assess the effectiveness of function-based nonlinear representation

Table 3 Result of ablation study

Model	Weibo						Twitter					
	0.5 h			1 h			2 h			1 day		
	MSLE	SMAPE		MSLE	SMAPE		MSLE	SMAPE		MSLE	SMAPE	
w/o time flow	2.265	0.232		2.173	0.266		1.967	0.290		6.063	0.585	
w/o ReLU-based multi-scale linear attention	2.131	0.219		2.090	0.248		1.871	0.278		5.827	0.563	
w/o KAN	2.315	0.234		2.169	0.250		1.953	0.285		6.093	0.572	
CasMST-KAN	2.080	0.219		2.061	0.243		1.839	0.273		5.585	0.560	

Bold values indicate the best results among all compared methods

Table 4 Parameter analysis results on the Weibo dataset (2 h prediction)

Grid size	Spline order	Kernel size	MSLE	SMAPE
3	3	7	1.925	0.287
3	3	5	1.939	0.287
4	3	5	1.839	0.273
4	4	5	1.864	0.274
4	2	3	1.862	0.277
4	2	5	1.850	0.273
5	3	5	1.911	0.283
5	2	3	1.899	0.281

Bold values indicate the best results among all compared methods

Table 5 ReLU multi-scale linear attention configurations on the Weibo dataset (1 h prediction)

Scale configuration	MSLE	SMAPE
(3, 3)	2.090	0.221
(3, 5)	2.099	0.219
(3, 7)	2.106	0.223
(5, 3)	2.130	0.227
(5, 5)	2.093	0.223
(5, 7)	2.080	0.219
(7, 3)	2.094	0.219
(7, 5)	2.146	0.226
(7, 7)	2.143	0.223

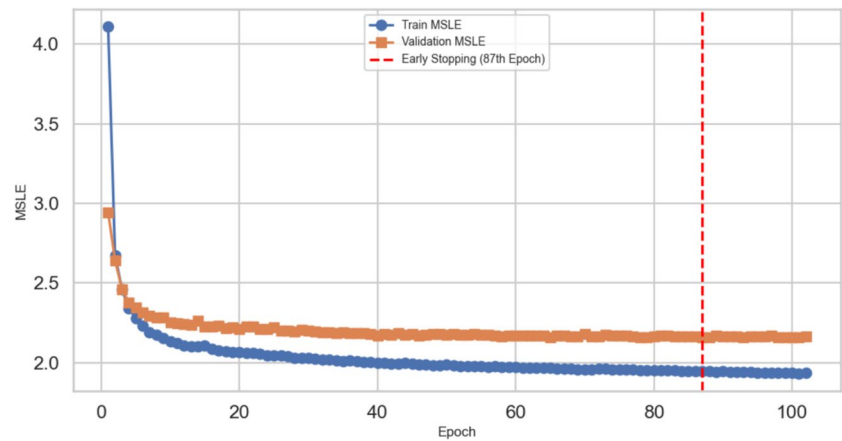
Bold values indicate the best results among all compared methods

compared to conventional feedforward neural networks that lack such expressive capacity.

The ablation study results reveal the importance of each component in the CasMST-KAN model:

- *Without time flow structure (MSTC and LSTM):* Removing the time modeling modules significantly degrades performance across all metrics and datasets. This is because the model loses its ability to capture the sequential and multi-scale dynamics of information diffusion, which are crucial for understanding how cascades evolve over time.
- *Without ReLU-based multi-scale linear attention:* Excluding the ReLU-based multi-scale linear attention module causes a moderate drop in accuracy. This component enables efficient interaction between structural and temporal features. Without it, the model cannot effectively highlight important cross-dimensional dependencies, leading to less informative representations.
- *Without KAN:* Replacing the Kolmogorov–Arnold network (KAN) with a standard MLP results in consistent performance degradation. KAN employs learnable univariate functional mappings on edges to better approximate complex nonlinear relationships between nodes. Removing KAN weakens the model's capability to capture high-order dependencies and intricate functional patterns critical for cascade size prediction.

Fig. 3 Training and validation MSLE for Weibo (0.5 h prediction)



These observations highlight the necessity of each component in CasMST-KAN. The full model achieves the best performance by jointly leveraging temporal patterns, structural attention, and cross-feature interaction.

5.6 Parameter analysis

We conducted a parameter analysis to investigate the effects of key hyperparameters on the performance of CasMST-KAN. Specifically, Table 4 presents the results of varying the kernel size in the multi-scale temporal convolution (MSTC) module, the grid size in the kernel attention network (KAN), and the spline order. These components are crucial for capturing temporal and structural dependencies. Meanwhile, Table 5 shows the performance under different configurations of the ReLU-based multi-scale linear attention mechanism, where we adjust the number of scales to examine its impact on local feature extraction and multi-scale learning.

MSTC kernel size: The kernel size was tested with values of 3, 5, and 7, with a kernel size of 5 yielding the best performance. A kernel size of 3 captured overly localized patterns, potentially missing critical temporal trends relevant to cascade prediction. In contrast, a kernel size of 7 introduced excessive temporal coverage, diluting meaningful local information and increasing the risk of overfitting. The selected kernel size of 5 strikes a balance by capturing both short- and medium-term temporal dependencies, which aligns well with the hierarchical nature of cascade propagation.

KAN grid size and spline order: We evaluated grid sizes and spline orders around the chosen optimal values of 4 and 3, respectively. Smaller grid sizes or lower spline orders resulted in a loss of flexibility in modeling fine-grained attention patterns, while larger values led to overparameterization and degraded performance. The chosen parameters enable CasMST-KAN to capture intricate relationships

between nodes without introducing excessive computational overhead, thus effectively supporting cascade prediction.

ReLU-based multi-scale linear attention scales: The scales parameter, which controls the number of receptive fields in ReLU-based multi-scale linear attention, was analyzed with values of (3, 3), (3, 5), (5, 7), (7, 9), and (9, 9). The configuration (5, 7) provided the best balance between capturing multiscale patterns and maintaining computational efficiency. However, experimental results showed that as long as the scale values were not excessively small (e.g., (3, 3)) or excessively large (e.g., (9, 9)), the performance did not vary significantly. This suggests that ReLU-based Multi-Scale Linear Attention is relatively robust to moderate changes in scales. A possible explanation for this phenomenon is that the module inherently adjusts its attention weights to focus on the most informative regions, reducing sensitivity to minor variations in the scale parameters. Additionally, this robustness might be attributed to the complementary effects of other components in CasMST-KAN, which compensate for suboptimal scale choices under certain conditions.

5.7 Training and overfitting mitigation

Training CasMST-KAN involved careful optimization of hyperparameters to balance convergence speed and generalization performance. We analyzed the effects of learning rate and weight decay on training stability and overfitting mitigation.

Learning rate: During experimentation, commonly used learning rates such as 0.001 and 0.0008 often caused CasMST-KAN to converge prematurely on the Weibo dataset, resulting in suboptimal generalization. A lower learning rate, optimized using Optuna, allowed CasMST-KAN to converge more gradually over at least 50 epochs, leading to improved performance and better alignment with validation loss trends.

Weight decay: To control overfitting, weight decay values were systematically tested in the range of $[10^{-5}, 10^{-3}]$. Higher weight decay values ($\geq 10^{-3}$) excessively penalized CasMST-KAN's complexity, limiting its ability to capture subtle patterns in cascade dynamics. The optimal weight decay value, tuned via Optuna, provided a regularization effect that suppressed overfitting while preserving CasMST-KAN's capacity for intricate pattern learning.

Early stopping: To further enhance generalization, we employed early stopping with a patience of 15 epochs. Training was halted if no improvement in validation loss was observed over these 15 epochs. This strategy ensured CasMST-KAN avoided overtraining and maintained robust performance across both datasets.

Due to the above efforts, including Optuna-optimized learning rate and weight decay, as well as the early stopping mechanism, our model effectively mitigated overfitting, as evident in the experimental results presented below (Fig. 3). It can be observed that early stopping was applied at the 87th epoch. While the training set's MSLE continued to decrease afterward, the validation set's MSLE did not improve, effectively avoiding overfitting.

6 Conclusions

We proposed CasMST-KAN, a novel deep learning framework for predicting the growth scale of information cascades by jointly modeling multi-scale temporal patterns and structural dependencies. CasMST-KAN integrates multi-scale temporal convolution, LSTM, Kolmogorov–Arnold networks, and ReLU-based multi-scale linear attention to effectively capture hierarchical dynamics and contextual information. Experimental results on real-world datasets (Weibo and Twitter) demonstrate the superior performance of CasMST-KAN over state-of-the-art baselines in both accuracy and efficiency.

In future work, we plan to explore more efficient graph sampling strategies and refine the integration of temporal and structural features. We also aim to extend the framework to handle multi-modal data for broader applicability.

Acknowledgements This work is supported by the National Natural Science Foundation of China under Grants (61907021).

Author contributions D.X. contributed to conceptualization, methodology, formal analysis, writing—original draft, and supervision. Z.Z. contributed to data curation, software development, validation, visualization, and writing—review and editing. D.C. contributed to investigation, resource provision, and writing—review and editing. Y.T. contributed to project administration, funding acquisition, and writing—review and editing. All authors reviewed the manuscript and approved its final version for publication.

Data availability No datasets were generated or analysed during the

current study.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Aiello LM, Petkos G, Martin C et al (2013) Sensing trending topics in twitter. *IEEE Trans Multimed* 15(6):1268–1282
2. Cai H, Li J, Hu M et al (2022) Efficientvit: multi-scale linear attention for high-resolution dense prediction. [arXiv:2205.14756](https://arxiv.org/abs/2205.14756)
3. Cao Q, Shen H, Cen K et al (2017) Deephawkes: bridging the gap between prediction and understanding of information cascades. In: *Proceedings of the 2017 ACM on conference on information and knowledge management*, pp 1149–1158
4. Cao Q, Shen H, Gao J et al (2020) Popularity prediction on social platforms with coupled graph neural networks. In: *Proceedings of the 13th international conference on web search and data mining*, pp 70–78
5. Chen X, Zhou F, Zhang K et al (2019) Information diffusion prediction via recurrent cascades convolution. In: *2019 IEEE 35th international conference on data engineering (ICDE)*. IEEE, pp 770–781
6. Cheng G, Yan X, Gao S et al (2023) Cassampling: exploring efficient cascade graph learning for popularity prediction. In: *Joint European conference on machine learning and knowledge discovery in databases*. Springer, pp 70–86
7. Cheng J, Adamic L, Dow PA et al (2014) Can cascades be predicted? In: *Proceedings of the 23rd international conference on world wide web*, pp 925–936
8. Cheng Z, Zhou F, Xu X et al (2024) Information cascade popularity prediction via probabilistic diffusion. *IEEE Trans Knowl Data Eng* 36(12):8541–55
9. Chung J, Gulcehre C, Cho K et al (2014) Empirical evaluation of gated recurrent neural networks on sequence modeling. [arXiv:1412.3555](https://arxiv.org/abs/1412.3555)
10. Fei W, Gu J, Yang Y et al (2015) Credit risk evaluation based on social media. *Proc Comput Sci* 55:725–731
11. Gao J, Shen H, Liu S et al (2016) Modeling and predicting retweeting dynamics via a mixture process. In: *Proceedings of the 25th international conference companion on world wide web*, pp 33–34
12. Gao S, Ma J, Chen Z (2014) Effective and effortless features for popularity prediction in microblogging network. In: *Proceedings of the 23rd international conference on world wide web*, pp 269–270
13. Hochreiter S (1997) Long short-term memory. *Neural Comput* 9:1735–80
14. Huang C, Xu H, Xu Y et al (2021) Knowledge-aware coupled graph neural network for social recommendation. In: *Proceedings of the AAAI conference on artificial intelligence*, pp 4115–4122
15. Huang P, Guo L, Zhong Y (2022) Efficient algorithms for maximizing group influence in social networks. *Tsinghua Sci Technol* 27(5):832–842
16. Huang P, Guo L, Zhong Y (2022) Efficient algorithms for maximizing group influence in social networks. *Tsinghua Sci Technol* 27(5):832–842
17. Jia X, Shang J, Liu D et al (2022) Hedon: heterogeneous diffusion attention network for popularity prediction of online content. *Knowl-Based Syst* 254:109659

18. Jiao P, Song W, Wang Y et al (2025) Cascade popularity prediction: a multi-view learning approach with socialized modeling. *IEEE Trans Netw Sci Eng* 12(2):1198–209
19. Kempe D, Kleinberg J, Tardos É (2003) Maximizing the spread of influence through a social network. In: *Proceedings of the ninth ACM SIGKDD international conference on knowledge discovery and data mining*, pp 137–146
20. Kempe D, Kleinberg J, Tardos É (2003) Maximizing the spread of influence through a social network. In: *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, pp 137–146
21. Kipf TN, Welling M (2016) Semi-supervised classification with graph convolutional networks. [arXiv:1609.02907](https://arxiv.org/abs/1609.02907)
22. Liao D, Xu J, Li G et al (2019) Popularity prediction on online articles with deep fusion of temporal process and content features. In: *Proceedings of the AAAI conference on artificial intelligence*, pp 200–207
23. Liu Z, Huang X, Zhang J et al (2024) Multivariate time-series anomaly detection based on enhancing graph attention networks with topological analysis. In: *Proceedings of the 33rd ACM international conference on information and knowledge management*, pp 1555–1564
24. Liu Z, Wang Y, Vaidya S, et al (2024) Kan: Kolmogorov–Arnold networks. [arXiv:2404.19756](https://arxiv.org/abs/2404.19756)
25. Lymperopoulos IN (2016) Predicting the popularity growth of online content: model and algorithm. *Inf Sci* 369:585–613
26. Manouchehri MA, Helfroush MS, Danyali H (2021) Temporal rumor blocking in online social networks: a sampling-based approach. *IEEE Trans Syst Man Cybern Syst* 52(7):4578–4588
27. Martin T, Hofman JM, Sharma A et al (2016) Exploring limits to prediction in complex social systems. In: *Proceedings of the 25th international conference on world wide web*, pp 683–694
28. Mishra S, Rizoïu MA, Xie L (2016) Feature driven and point process approaches for popularity prediction. In: *Proceedings of the 25th ACM international on conference on information and knowledge management*, pp 1069–1078
29. Shen H, Wang D, Song C et al (2014) Modeling and predicting popularity dynamics via reinforced Poisson processes. In: *Proceedings of the AAAI conference on artificial intelligence*
30. Sun X, Zhou J, Liu L et al (2023) Explicit time embedding based cascade attention network for information popularity prediction. *Inf Process Manag* 60(3):103278
31. Sun X, Zhou J, Liu L et al (2023) Castformer: a novel cascade transformer towards predicting information diffusion. *Inf Sci* 648:119531
32. Ta N, Li K, Yang Y et al (2020) Evaluating public anxiety for topic-based communities in social networks. *IEEE Trans Knowl Data Eng* 34(3):1191–1205
33. Tang X, Liao D, Huang W et al (2021) Fully exploiting cascade graphs for real-time forwarding prediction. In: *Proceedings of the AAAI conference on artificial intelligence*, pp 582–590
34. Velickovic P, Cucurull G, Casanova A et al (2017) Graph attention networks. *Stat* 1050(20):10–48550
35. Weng L, Menczer F, Ahn YY (2013) Virality prediction and community structure in social networks. *Sci Rep* 3(1):1–6
36. Wu Z, Zhou J, Liu L et al (2022) Deep popularity prediction in multi-source cascade with heri-gcn. In: *2022 IEEE 38th international conference on data engineering (ICDE)*. IEEE Computer Society, pp 1714–1726
37. Xiao C, Liu C, Ma Y et al (2020) Time sensitivity-based popularity prediction for online promotion on twitter. *Inf Sci* 525:82–92
38. Xu X, Zhou F, Zhang K et al (2021) Casflow: exploring hierarchical structures and propagation uncertainty for cascade prediction. *IEEE Trans Knowl Data Eng* 35(4):3484–3499
39. Xu Z, Qian M, Huang X et al (2020) Casgcn: predicting future cascade growth based on information diffusion graph. [arXiv:2009.05152](https://arxiv.org/abs/2009.05152)
40. Xue G, Liu S, Ren L et al (2022) Forecasting the subway passenger flow under event occurrences with multivariate disturbances. *Expert Syst Appl* 188:116057
41. Zhao Q, Erdogdu MA, He HY et al (2015) Seismic: a self-exciting point process model for predicting tweet popularity. In: *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, pp 1513–1522
42. Zhou F, Xu X, Trajcevski G et al (2021) A survey of information cascade analysis: models, predictions, and recent advances. *ACM Comput Surv (CSUR)* 54(2):1–36
43. Zhu J, Ghosh S, Wu W (2021) Robust rumor blocking problem with uncertain rumor sources in social networks. *World Wide Web* 24:229–247

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.